API Katalog Marketplace

Dokumentasi ini ditujukan untuk developer web frontend dan Android.

Konfigurasi dasar

Base URL API:

```
{BASE_URL}/api/v1
```

Contoh lokal:

```
http://127.0.0.1:8001/api/v1
```

Endpoint katalog bersifat **public read-only** dan tidak memerlukan token autentikasi.

Header yang disarankan untuk setiap request:

```
Accept: application/json
```

Katalog gabungan produk dan bundle

```
GET /catalog
```

Endpoint ini digunakan untuk halaman beranda, eksplorasi, dan pencarian marketplace. Response menggabungkan listing produk biasa dan listing paket bundle.

Query parameter

Parameter	Tipe	Wajib	Keterangan
`q`	string	Tidak	Pencarian token-AND: tiap kata dipisah spasi, semua kata harus cocok (boleh beda urutan & beda field). Mencari nama, kode/SKU, nama varian, nama kategori aktif, nama/SKU produk komponen bundle, deskripsi etalase toko, atau nama toko. Maksimal 100 karakter, maksimal 10 kata.
`type`	string	Tidak	`product` atau `bundle`. Jika dikosongkan, keduanya ditampilkan.
`store`	string	Tidak	Filter berdasarkan slug toko.

Parameter	Tipe	Wajib	Keterangan
`category`	string	Tidak	Filter slug kategori. Untuk bundle, kategori dicari dari produk komponennya.
`min_price`	number	Tidak	Harga minimum.
`max_price`	number	Tidak	Harga maksimum.
`sort`	string	Tidak	`relevance`, `latest`, `name_asc`, `name_desc`, `price_asc`, atau `price_desc`. Default: `relevance` saat `q` terisi, `latest` saat kosong.
`page`	integer	Tidak	Nomor halaman eksplisit. Default: `1`.
`per_page`	integer	Tidak	Jumlah data per halaman, minimal 1 dan maksimal 50. Default: `15`.

Contoh request:

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?
q=lambung&type=bundle&sort=price_asc&per_page=20' \
  --header 'Accept: application/json'
```

Response berhasil


```
{
  "success": true,
  "message": "Katalog gabungan berhasil diambil.",
  "data": [
    {
      "type": "product",
      "listing_id": 12,
      "id": 5,
      "name": "Sehat Lambungku",
      "slug": "sehat-lambungku",
      "code": "PRD-WAL-ABCDEFGH",
      "price": {
        "min": "50000.00",
        "max": "75000.00"
      },
      "image_url": "http://127.0.0.1:8001/storage/12/image.jpg",
      "store": {
        "id": 3,
        "name": "Walatra Herbal",
        "slug": "walatra-herbal",
        "city": "Bandung",
        "province": "Jawa Barat"
      },
      "published_at": "2026-09-17T10:00:00+07:00",
      "matched_in": ["name"]
    },
```

```
      {
        "type": "bundle",
        "listing_id": 8,
        "id": 2,
        "name": "Paket Lambung Sehat",
        "slug": "paket-lambung-sehat",
        "code": "BND-ABCDEFGH",
        "price": {
          "value": "120000.00"
        },
        "image_url": null,
        "store": {
          "id": 3,
          "name": "Walatra Herbal",
          "slug": "walatra-herbal",
          "city": "Bandung",
          "province": "Jawa Barat"
        },
        "published_at": "2026-09-17T10:00:00+07:00",
        "matched_in": []
      }
    ],
    "links": {
      "first": "http://127.0.0.1:8001/api/v1/catalog?page=1",
      "last": "http://127.0.0.1:8001/api/v1/catalog?page=2",
      "prev": null,
      "next": "http://127.0.0.1:8001/api/v1/catalog?page=2"
    },
    "meta": {
      "current_page": 1,
      "from": 1,
      "last_page": 2,
      "per_page": 15,
      "to": 15,
      "total": 30
    }
  }
```


Frontend atau Android wajib menggunakan field `type` untuk menentukan halaman detail:

```
type = product -> /catalog/products/{slug}
type = bundle  -> /catalog/bundles/{slug}
```

`listing_id` adalah ID penawaran produk/bundle pada toko. `id` adalah ID master produk atau master bundle.

Contoh penggunaan filter

```
# Semua item katalog terbaru
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog' \
  --header 'Accept: application/json'

# Hanya produk dari kategori tertentu
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?
type=product&category=pencernaan&sort=price_asc' \
  --header 'Accept: application/json'

# Hanya bundle pada toko tertentu dengan rentang harga
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?type=bundle&store=walatra-
herbal&min_price=50000&max_price=250000' \
  --header 'Accept: application/json'

# Halaman kedua dengan 20 item
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?page=2&per_page=20' \
  --header 'Accept: application/json'
```

Contoh pencarian (token-AND + relevance)

Pencarian dipecah per kata, semua kata wajib cocok, boleh beda urutan dan beda field. Contoh: `kronis lambung` tetap menemukan produk bernama `Madu Asli Sumbawa` yang deskripsinya `Untuk kesehatan lambung kronis`.

```
# 1. Urutan kata tidak penting (token-AND)
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=kronis%20lambung' \
  --header 'Accept: application/json'
# -> data[0].name = "Madu Asli Sumbawa" (kronis ada di deskripsi, lambung
ada di nama)

# 2. Cari berdasarkan SKU varian
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=SKU-UNIK-12345' \
  --header 'Accept: application/json'
# -> data[0].name = produk pemilik SKU tersebut

# 3. Cari berdasarkan kata di deskripsi etalase toko
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=lambung%20kronis' \
  --header 'Accept: application/json'

# 4. Cari berdasarkan nama varian (misal kemasan)
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=100ml' \
  --header 'Accept: application/json'
```

```
# -> menemukan "Herbal Lambung X" yang variannya bernama "Kemasan 100ml"

# 5. Cari berdasarkan nama kategori
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=pencernaan' \
  --header 'Accept: application/json'

# 6. Cari bundle dari nama produk komponennya
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=Jahe%20Merah&type=bundle' \
  --header 'Accept: application/json'
# -> data[0].name = "Paket Sehat Keluarga" (isinya "Jahe Merah Premium")

# 7. Relevance eksplisit: nama persis selalu paling depan
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?
q=Madu%20Lambung&sort=relevance' \
  --header 'Accept: application/json'
# -> data[0].name = "Madu Lambung" (exact match, skor 100),
#    bukan "Teh Herbal Madu Lambung Campur" (contains, skor 50)

# 8. Tanpa sort + ada q = otomatis relevance
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=Propolis' \
  --header 'Accept: application/json'
```

Skor relevance 0-100 dihitung dari: nama persis 100, awalan nama 80, mengandung 50, kode persis 70, SKU/komponen 20, nama toko 10, ada deskripsi 8, ada gambar 7. Tanpa `q`, skor semua 0 sehingga `sort` default menjadi `latest`.

Contoh respons pencarian + `matched_in`

Request:

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog?q=Hutan%20Kalimantan' \
  --header 'Accept: application/json'
```

Respons (dipotong):

```
{
  "success": true,
  "message": "Katalog gabungan berhasil diambil.",
  "data": [
    {
      "type": "product",
      "listing_id": 21,
      "id": 9,
      "name": "Madu Hutan Kalimantan",
```

```
      "slug": "madu-hutan-kalimantan",
      "code": "PRD-WAL-1A2B3C4D",
      "price": { "min": "93000.00", "max": "93000.00" },
      "image_url": "http://127.0.0.1:8001/storage/21/image.jpg",
      "store": {
        "id": 3,
        "name": "Walatra Herbal",
        "slug": "walatra-herbal",
        "city": "Bandung",
        "province": "Jawa Barat"
      },
      "published_at": "2026-09-17T10:00:00+07:00",
      "matched_in": ["name"]
    }
  ]
}
```

Contoh penggunaan di web frontend

```
// 1. Homepage: tanpa q = otomatis latest
const homeRes = await fetch(`${BASE_URL}/api/v1/catalog?per_page=20`, {
  headers: { Accept: 'application/json' },
});

// 2. Searchbar: ketik "kronis lambung" (urutan bebas)
const params = new URLSearchParams({
  q: 'kronis lambung',
  sort: 'relevance', // boleh dihilangkan, default saat ada q = relevance
  page: '1',
  per_page: '20',
});

const response = await fetch(`${BASE_URL}/api/v1/catalog?${params}`, {
  headers: { Accept: 'application/json' },
});

const result = await response.json();

if (!result.success) {
  throw new Error(result.message);
}

result.data.forEach((item) => {
  const detailUrl = item.type === 'product'
    ? `/produk/${item.slug}`
    : `/bundle/${item.slug}`;

  // Label "ketemu di ..." dari field matched_in (non-breaking, bisa
kosong)
  const foundIn = (item.matched_in ?? []).join(', ') || 'katalog';
```

```
  console.log(item.name, item.type, detailUrl, `ketemu di ${foundIn}`);
});
```

```
// 3. Pagination: halaman 1 vs 2 harus beda listing_id
const page1 = await (await fetch(`${BASE_URL}/api/v1/catalog?
type=product&per_page=2&page=1&sort=name_asc`, {
  headers: { Accept: 'application/json' },
})).json();

const page2 = await (await fetch(`${BASE_URL}/api/v1/catalog?
type=product&per_page=2&page=2&sort=name_asc`, {
  headers: { Accept: 'application/json' },
})).json();

console.log(page2.meta.current_page); // 2
console.log(page1.data[0].listing_id !== page2.data[0].listing_id); // true
```

Gunakan `links.next` atau `meta.current_page` dan `meta.last_page` untuk pagination. Jangan menebak URL halaman berikutnya sendiri jika `links.next` bernilai `null`.

Field `matched_in` hanya terisi bermakna saat parameter `q` diisi. Nilainya subset dari `name`, `code`, atau `store` yang cocok dengan seluruh kata kunci. Contoh: `q=Hutan Kalimantan` menghasilkan `"matched_in": ["name"]`. Tanpa `q`, nilainya `[]`. Field ini bersifat tambahan (non-breaking) agar frontend bisa menampilkan label "ketemu di nama/toko" tanpa parsing manual. Keterbatasan saat ini: kecocokan di deskripsi, varian, kategori, atau komponen bundle tetap membuat item muncul di hasil, tetapi tidak mengubah nilai `matched_in`.

```
// 4. Android (Kotlin + Retrofit): search + pagination
interface CatalogApi {
    @GET("catalog")
    suspend fun searchCatalog(
        @Query("q") query: String, // misal "100ml" atau "pencernaan"
        @Query("sort") sort: String = "relevance",
        @Query("page") page: Int = 1,
        @Query("per_page") perPage: Int = 20,
    ): CatalogResponse
}

// Pemakaian:
// val result = api.searchCatalog(query = "kronis lambung")
// val label = result.data.firstOrNull()?.matchedIn?.joinToString() ?:
"katalog"
```

Katalog produk

```
GET /catalog/products
GET /catalog/products/{slug}
```

Endpoint list produk mendukung `q`, `store`, `category`, `min_price`, `max_price`, `sort`, `page`, dan `per_page`. Nilai `sort` yang tersedia:

```
relevance, latest, name_asc, name_desc, price_asc, price_desc
```

Response list menggunakan struktur `listing_id`, `product`, `store`, `variants`, `images`, dan `published_at`.

Contoh request list dan detail:

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/products?
q=propolis&sort=relevance' \
  --header 'Accept: application/json'

curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/products/propolis-sm' \
  --header 'Accept: application/json'
```

Katalog bundle

```
GET /catalog/bundles
GET /catalog/bundles/{slug}
```

Endpoint list bundle mendukung `q`, `store`, `sort`, `page`, dan `per_page`. Pencarian `q` menggunakan token-AND seperti katalog gabungan dan mencari nama bundle, kode, nama toko, deskripsi etalase, SKU/nama varian komponen, serta nama produk komponen. Nilai `sort` yang tersedia:

```
latest, name_asc, name_desc, price_asc, price_desc
```

Setiap bundle memiliki daftar item produk pada field `bundle.items`.

Contoh request list dan detail:

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/bundles?
sort=price_asc&per_page=10' \
  --header 'Accept: application/json'
```

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/bundles/paket-lambung-sehat'
\
  --header 'Accept: application/json'

# Cari bundle dari nama produk komponennya (token-AND, konsisten dengan
/catalog)
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/bundles?q=Jahe%20Merah' \
  --header 'Accept: application/json'

# Cari bundle dari SKU varian komponen
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/bundles?q=SKU-TEST-BUNDLE-01'
\
  --header 'Accept: application/json'
```

Katalog kategori

```
GET /catalog/categories
GET /catalog/categories/{slug}
```

Kategori hanya dikelola oleh superadmin. Produk dapat memiliki lebih dari satu kategori. Endpoint kategori hanya menghitung dan menampilkan produk katalog yang memenuhi aturan aktif.

Contoh request:

```
curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/categories' \
  --header 'Accept: application/json'

curl --request GET \
  --url 'http://127.0.0.1:8001/api/v1/catalog/categories/pencernaan' \
  --header 'Accept: application/json'
```

Aturan visibilitas katalog

Data hanya dikembalikan jika memenuhi aturan berikut:

- Toko berstatus `active`.
- Listing toko berstatus `active`.
- Produk berstatus `active` dan memiliki minimal satu varian aktif.
- Bundle berstatus `active` dan memiliki item bundle.
- Seluruh varian dan produk yang digunakan bundle berstatus aktif.
- Data yang sudah di-soft-delete tidak ditampilkan.

Client tidak perlu mengubah status atau melakukan validasi availability sendiri. Jika status berubah di backend, data tersebut tidak akan dikembalikan oleh endpoint katalog.

Error response

Format error menggunakan struktur standar berikut:

```
{
  "success": false,
  "message": "Tipe katalog hanya boleh product atau bundle.",
  "errors": {
    "type": [
      "Tipe katalog hanya boleh product atau bundle."
    ]
  }
}
```

HTTP status yang umum:

Status	Keterangan
200	Request berhasil.
404	Produk atau bundle tidak ditemukan/tidak aktif.
422	Parameter query tidak valid.
429	Terlalu banyak request dalam waktu singkat.
500	Kesalahan internal server. Client dapat mencoba kembali kemudian.